

Frontend Developer Task - Test Management Application

Overview

Build a test management application that allows users to create tests, add questions, and publish them. This is a 5-page flow application focusing on CRUD operations and API integration.

Application Flow

Page 1: Login Page

- Simple login form with userId and password fields
- Form validation
- JWT token management (store in localStorage/sessionStorage)
- Redirect to dashboard on successful login
- Error handling for failed login attempts

Page 2: Dashboard / Test List

- Display all tests in a table/card layout
- Show test details: name, subject, status, created date
- Actions: Edit, View, Delete buttons
- "Create New Test" button
- Filter/search functionality (bonus)

Page 3: Create/Edit Test Page

- Form fields:
 - Test Name (required)
 - Subject (dropdown - fetched from API)
 - Test Type (dropdown/select)
 - Topics (multi-select based on selected subject)
 - Sub-topics (multi-select based on selected topics)
 - Difficulty level
 - Marking scheme: correct_marks, wrong_marks, unattempt_marks
 - Total time, Total marks
- Save as Draft functionality
- "Next: Add Questions" button
- Form validation

Page 4: Add Questions Page

- Display selected test details at top
- Form to add questions:
 - Question text
 - 4 options (option1, option2, option3, option4)
 - Correct option
 - Explanation (optional)
 - Difficulty (optional)
 - Topic and Sub-topic (optional, dropdowns)
 - Media URL (optional)
- "Add Another Question" button
- List of added questions with edit/delete actions
- "Save & Continue" button
- Minimum 1 question required

Page 5: Preview & Publish

- Display complete test overview:
 - Test details
 - All questions with options

- Edit test or questions buttons
- "Publish Test" button
- Success message on publish
- Redirect to dashboard

Authentication

All APIs except login require JWT token in header:

```

Authorization: Bearer <token>

```

1. Login

****POST**** `/auth/login`

```json

```
{
 "userId": "string",
 "password": "string"
}
```

**\*\*Response:\*\***

```json

```
{
  "success": true,
  "data": {
    "token": "jwt-token-here",
    "user": { ... }
  }
}
```

2. Get All Subjects

****GET**** `/subjects`

****Response:****

```json

```
{
 "success": true,
 "data": [
 {
 "id": "uuid",
 "name": "Mathematics"
 }
]
}
```

### ### 3. Get Topics by Subject

**\*\*GET\*\*** `/topics/subject/:subjectId`

**\*\*Response:\*\***

```json

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "Algebra",
      "subject_id": "uuid"
    }
  ]
}
```

```
    }  
  ]  
}  
...
```

```
### 4. Get Sub-topics by Topic  
**GET** `/sub-topics/topic/:topicId`  
**Response:**  
```json  
{
 "success": true,
 "data": [
 {
 "id": "uuid",
 "name": "Linear Equations",
 "topic_id": "uuid"
 }
]
}
}```
```

```
5. Get All Tests
GET `/tests`
Response:
```json  
{  
  "success": true,  
  "data": [  
    {  
      "id": "uuid",  
      "name": "Sample Test",  
      "subject": "Mathematics",  
      "topics": ["Algebra", "Geometry"],  
      "status": "draft",  
      "created_at": "2025-01-15T10:00:00Z"  
    }  
  ]  
}  
}```
```

```
### 6. Create Test  
**POST** `/tests`  
```json  
{
 "name": "Sample Test",
 "type": "chapterwise",
 "subject": "subject-uuid",
 "topics": ["topic-uuid-1", "topic-uuid-2"],
 "sub_topics": ["subtopic-uuid-1"],
 "correct_marks": 4,
 "wrong_marks": -1,
 "unattempt_marks": 0,
 "difficulty": "medium",
 "total_time": 60,
 "total_marks": 250,
 "total_questions": 50
 "status": null
}
```

```
...
Response:
```json
{
  "success": true,
  "data": {
    "id": "test-uuid",
    "name": "Sample Test",
    ...
  },
  "message": "Test created successfully"
}
...

```

```
### 7. Update Test
**PUT** `/tests/:id`
```json
{
 "name": "Updated Test Name",
 "questions": ["question-uuid-1", "question-uuid-2"],
 "total_questions": 10,
 "total_marks": 40
}
...

```

```
8. Get Test by ID
GET `/tests/:id`
Response:
```json
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "Sample Test",
    "subject": "Mathematics",
    "topics": ["Algebra"],
    "questions": ["q1-uuid", "q2-uuid"],
    ...
  }
}
...

```

```
### 9. Bulk Create Questions
**POST** `/questions/bulk`
```json
{
 "questions": [
 {
 "type": "mcq",
 "question": "What is 2 + 2?",
 "option1": "3",
 "option2": "4",
 "option3": "5",
 "option4": "6",
 "correct_option": "option2",
 "explanation": "Basic addition",
 "difficulty": "easy",
 "test_id": "test-uuid"
 }
]
}

```

```

 }
]
}
...
Response:
```json
{
  "success": true,
  "data": [
    {
      "id": "question-uuid",
      ...
    }
  ],
  "message": "Successfully created 1 questions"
}
...

### 10. Publish Test
**PUT** `/tests/:id`
```json
{
 "status": "live"
}
...

11. Sub Topic by Topic List
POST `/sub-topics/multi-topics`
```json
{
  "topicIds": [
    "24f22e65-7117-4242-aa9b-2d55021e5b3d",
    "24f22e65-7117-4242-aa9b-2d55021e5b3d"
  ]
}
...

### 12. Fetch Bulk
**POST** `/questions/fetchBulk`
```JSON
{
 "question_ids": ["50162cb5-8e0c-4760-bf8f-a80f50751119", "6b3e7eac-c8d6-4226-8765-f2a1c50de1f3"]
}
...

```

Good luck! We're excited to see your work.